CS570 Fall 2025: Analysis of Algorithms Exam III

	Points		Points
Problem 1	18	Problem 5	14
Problem 2	6	Problem 6	18
Problem 3	16	Problem 7	16
Problem 4	12		
			Total 100

First name	
Last Name	
Student ID	

Instructions:

1. This is a 2-hr exam. Closed book and notes. No electronic devices or internet access.
2. A single double sided 8.5in x 11in cheat sheet is allowed.
3. If a description to an algorithm or a proof is required, please limit your description or proof to within 150 words, preferably not exceeding the space allotted for that question.
4. No space other than the pages in the exam booklet will be scanned for grading.
5. If you require an additional page for a question, you can use the extra page provided within this booklet. However please indicate clearly that you are continuing the solution on the additional page.
6. Do not detach any sheets from the booklet.
7. If using a pencil to write the answers, make sure you apply enough pressure, so your answers are readable in the scanned copy of your exam.
8. Do not write your answers in cursive scripts.
9. This exam is printed double sided. Check and use the back of each page.

1) (18 points)
Mark the following statements as **TRUE** or **FALSE** by circling the correct answer. No need to provide any justification. (2 points each)

[TRUE/FALSE]
If P = NP, then the decision version of the Shortest Path problem is NP-hard.

[TRUE/FALSE]
Suppose $Y \leq_p X$ and there exists a 2-approximation for X, then there must exist a ρ-approximation for Y for some constant ρ.

[TRUE/FALSE]
If P != NP, the minimum spanning tree problem is NP-complete.

[TRUE/FALSE]
If an NP-Complete problem, say Y, can be reduced to X and X belongs to NP. We can conclude that X is NP-Complete.

[TRUE/FALSE]
If a problem X is in NP, then Vertex Cover must be polynomial-time reducible to X.

[TRUE/FALSE]
If an undirected graph G=(V,E) has a Hamiltonian Cycle, then any DFS tree in G has a depth |V| - 1.

[TRUE/FALSE]
If there is a 1.5-approximation algorithm for the general Traveling Salesman Problem, then there exists a polynomial-time algorithm for 3-SAT.

[TRUE/FALSE]
Linear programming is at least as hard as the Max Flow problem.

[TRUE/FALSE]
The shortest-path problem in a graph with nonnegative weights can be formulated as a Linear Programming problem.

2) (6 points)
For each question below, select **all** correct answers! 3 points each. **No partial credit**.

I. Which of the following statements are true?

(A) If Integer linear programming can be used to solve a problem X, then X is NP-Hard.

(B) The feasible region of a linear program is always a convex region.

(C) In a linear program, an optimal solution may exist which is not at a vertex of the feasible region

(D) The Simplex algorithm is a polynomial-time algorithm for solving linear programs.

II. Assuming P ≠ NP, which of the following statements are true?

(A) There does not exist a polynomial-time algorithm for 3-SAT.

(B) Independent Set $\leq_p$ 3-SAT

(C) 3−SAT $\leq_p$ Max-Flow

(D) Max−Flow $\leq_p$ 3-SAT

3) (16 points)
A company makes three models of desks: an executive model, an office model and a student model. Building each desk takes time in the cabinet shop, the finishing shop and the crating shop as shown in the table below:

Type of desk	Cabinet shop	Finishing shop	Crating shop	Profit
Executive	2	1	1	150
Office	1	2	1	125
Student	1	1	.5	50
Available hours	16	16	10	

The available hours at the three shops and the profit from each model is shown in the table as well. How many of each type should they make to maximize profit? Use linear programming to formulate your solution. Assume that real numbers (non-integer) are acceptable in your solution.

Solution:
Start by defining your variables:
x = number of executive desks made
y = number of office desks made
z = number of student desks made

Maximize $P=150x+125y+50z$.

Subject to:
$2x + y + z \leq 16$ cabinet hours
$x + 2y + z \leq 16$ finishing hours
$x + y + .5z \leq 10$ crating hours
$x \geq 0, y \geq 0, z \geq 0$

Rubrics:

3 points to define the dec vars
3 points for the objective function
3 points for each of the first 3 constraints (9 total)
1 point for the non-negativity constraints

Check Gradescope for more rubric details.

4) (12 points)
You are given two decision problems L1 and L2 in NP. You are given that L1 $\leq_p$ L2. For each of the following statements, state whether it is true, or false, and justify your answers.
a) If L1 is NP-complete, then L2 is NP-complete.
b) If L2 is in P, then L1 is in P.
c) Suppose L2 is solvable in O(n). Then, L1 is also solvable in O(n).

Solution:

a) True. Since L1 is NP-complete, it is NP-Hard. The poly-time reducibility L1 $\leq_p$ L2 implies that L2 is NP-Hard. Since L2 is given in NP as well, L2 is NP-complete.

2 points for correct answer. 1 point for inferring NP-hardness from the reducibility. 1 point for using "L2 in NP" to infer NP-completeness by definition.

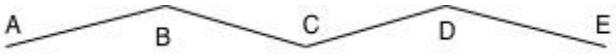
b) True. (2 points)
L1 $\leq_p$ L2 implies that we can solve L1 instances by using L2 black-box polynomial no. of times (and additional polynomial no. of steps) (1 point)
Since, L2 is in P, it has a poly-time algorithm/blackbox which allows L1 to have a poly-time algo as aforementioned (1 point)

c) False. (2 points)
The reduction L1 $\leq$p L2 only guarantees that we can solve L1 instances by using L2 black-box polynomial no. of times (and additional polynomial no. of steps). Thus, an O(n) algorithm for L2, if leveraged to solve L1 as above, could result in a much slower runtime than O(n), and thus, it is not possible to infer the existence of an O(n) algorithm for L1. (2 points)

5) 14 points

Recall the Minimum Vertex Cover (MVC) problem: For an undirected graph G = (V,E), a set of vertices S is a vertex cover if each edge in E has one of its endpoints in S. The MIS problem asks for the vertex cover with the minimum size.

We say S is an Excessive Vertex Cover (EVC) if it is possible to remove some vertex v from S to get a smaller vertex cover S\{v}.



For example, consider graph G above. Here, {B,D,E}, {A,B,D}, and {A,B,C,E} are some examples of excessive vertex covers, since we can remove a vertex in each case and obtain a smaller vertex cover. {A,C,E} is a vertex cover that is NOT excessive, since removing any single vertex does not give us a smaller vertex cover.

a) (6 points) Describe an efficient algorithm that given graph G, outputs a vertex cover S which is NOT excessive. (Hint: something very simple should suffice)

Take an arbitrary edge E and add one of the endpoints v to S. Delete v and all its edges. Repeat until the graph is empty.

Rubric:
6 points for any variant of the above, the key is to make sure that that any new vertex is included only if covers a previously uncovered edge)

Check Gradescope for more rubric details.

b) (8 points) Now, consider any such algorithm A that computes an **arbitrary** vertex cover that is not excessive (i.e., its output could be any vertex cover which is not excessive). We want to analyze if this can serve as a good approximation to the MVC problem.

Prove or disprove: There exists a constant $\alpha > 1$, s.t., A is guaranteed to output a vertex cover of size α*M(G) on every graph G, where M(G) denotes the size of a MVC in G.
To prove: provide a value of α and prove the approximation guarantee.
To disprove: Justify why no such α exists.

Consider a graph with one "central node" and n nodes connected to it. Choosing just the central node is a MVC of size 1. Choosing all the other nodes (except the central one) gives a vertex cover of size n which is not excessive. This is a ratio of n, so can choose arbitrarily large n to disprove any constant α.

Rubric:
+8pts (dis)Proof with counter-example and ratio $\lim \to 1/n=0$
+6pts (dis)Proof with counter-example but miss to state the ratio of approximation.

Check Gradescope for more rubric details.

6) (18 points) You are throwing a party for n people (including you). You decide to order pizzas for everyone. There are k different kinds of pizzas and you want to make sure you order at least one of each. Each pizza comes cut into 8 equal slices. Different pizzas may have different allergens (such as dairy, nuts etc.) and you know the information whether any person i can eat pizza of type j or not, denoted by binary parameter P_{ij} with value 1 or 0 respectively. You also know that everyone will eat at least 5 slices in total, but no more than 3 slices of any one kind. You want to decide the minimum number of pizzas to buy in total so that it is possible for everyone to eat subject to the constraints above. Assume you can serve specific pizza slices to people (subject to constraints above) since letting everyone choose freely may ruin your planning.
Formulate this problem as an integer linear program (ILP).

a) Define the decision variables of your ILP. (Describe what they represent in English)

b) What is the objective function in your ILP?

c) What are the constraints in your ILP?

X_{ij} = no. of slices friend i will eat of pizza kind j
N_j = no. of pizzas of kind j

Minimize $\sum_{j} N_j$

s.t.

$0 \le X_{ij} \le 3\ P_{ij} \quad \forall$ i = 1 to n, j = 1 to k (1)

$\sum_{j=1}^{k} X_{ij} \ge 5 \quad \forall$ i = 1 to n (2)

$\sum_{i=1}^{n} X_{ij} \le 8\ N_j \quad \forall$ j = 1 to k (3)

$N_j \ge 1 \quad \forall$ j = 1 to k (4)

$N_j \in Z \quad \forall$ j = 1 to k (5)

$X_{ij} \in Z \quad \forall$ i = 1 to n, j = 1 to k (6)

Rubrics:

a) 4 points to define the decision variables, 2 points if only xij are defined
b) 3 points for the objective function
c) 2 points for bounds on X_{ij} (lower bound 0, upper bound 3)
2 points for addressing allergen info P_{ij} (i.e. $X_{ij} \le 3\ P_{ij}$)
2 points for "5 slices per person" (constraint 2)
2 points to relate X_{ij} and N_j using '8 slices per pizza (constraint 3)
2 points for "at least 1 pizza of each kind (constraint 4)
1 point for the non-negativity integer constraints (constraint 5+6)

Check Gradescope for more rubric details.

7) Suppose we have N people. Suppose every pair (i,j) are either friends with each other or enemies, given by the relationship matrix R, where R[i,j] = 'F' or 'E' if (i,j) are friends or enemies respectively (Naturally, R[i,j] = R[j,i] and assume all entries R[i,i] are empty since self- relationships are undefined in this context).
A subset of 2k people is called a *Frienemy* group if it can be split into two groups P and Q of k people each, such that

- All people in P are friends with each other.
- All people in Q are friends with each other.
- Every i in P and every j in Q are enemies of each other.

The FRIENEMY problem takes no. of people N, relationship matrix R, and a non-negative integer k as input, and determines if there is a *Frienemy* group of 2k people as defined above. Prove that the FRIENEMY problem is

a) in NP. (4 points)

Certificate: 2k people split into groups P and Q of size k each (2 points)
Certifier: Check all k^2 pairs within P and k^2 pairs within Q that they are friends. Check all k^2 pairs with one person from P, Q each that they are enemies. (2 points)

b) NP-Hard. (Hint: Choose one of the following problems for your reduction: SAT/3SAT, independent set, Vertex/set cover, or k-clique) (12 points)

Reduce from Clique (or similarly from Ind. Set)

Construction: Given a Clique instance (G,k), add a clique of size k, not connected to G. Given this G', construct a FRIENEMY instance with a person constructed for each node in G', and R[i,j] = 'F' if (i,j) is an edge in G', otherwise R[i,j] = 'E'.
Claim: This instance has a Frienemy group of 2k if and only if G has a clique of size k.
Proof.
=>) If G has a k-Clique C, the people corresponding to C form one partition P of the frienemy group and the people corresponding to the newly added k vertices form the set Q of the frienemy group. All people within P and similarly within Q are indeed friends since the corresponding vertices form cliques, and any "P-Q pairs" are enemies since the vertices corresponding to Q were isolated from all of G, and thus, particularly from P.
<=) If there is a frienemy group with partition (P, Q), suppose the corresponding vertex-sets are S,T. Since all people in P are friends, S must be a clique, the same follows for T. But a clique cannot have a vertex from G and another newly added vertex since the new vertices are all disconnected from G. Thus, the cliques are either entirely in G or entirely outside. But only one of S, T can be outside G, since only k new vertices are added. Thus, at least one of them is in G, thus, G has a k-Clique.

Rubric:
Construction (7 points): Construct G' (4 points) + person for each node (1 point) + Relationships R from edges (2 points)
Claim (1 point)
Proof (4 points): 2 for each direction.

Check Gradescope for more rubric details.

Remarks:
1) Can simply take two copies of G to make G'.
2) Can reduce from IS similarly. (Main difference in construction: R is constructed opposite to the previous construction, i.e., R[i,j] = '**E**' if (i,j) is an edge in G', otherwise R[i,j] = '**F**'). Remark 1 applies for this reduction as well.

Additional Space

Additional Space

Additional Space